CS3000 Practice Quiz 5E — Answer Key

Mixed review: concepts, correctness, and unbounded knapsack

Notation follows the class notes on dynamic programming: $C(v)$ (coin change), $K(i, w)$ (0/1 knapsack), $L(i, j)$ (LCS), $D(i, j)$ (edit distance); strings and items are 1-indexed. Every table below was computed by a program and checked against brute force. Shaded cells mark the traceback path. Point splits imitate the Gradescope rubrics used on Quizzes 1–4 (each question 5 pts, 10 total).

Q1. Concepts

(a) *Overlapping subproblems* (subproblems share sub-subproblems) together with optimal substructure. Divide-and-conquer applies when the subproblems are independent, so there is nothing to reuse; DP stores each subproblem's answer once and reuses it.

(b) Calls satisfy $T(n) = 1 + T(n-1) + T(n-2)$, $T(0) = T(1) = 1$: $T(2..6) = 3, 5, 9, 15, 25$, so **25 calls**. Distinct subproblems: $F(0), \ldots, F(6)$, i.e. **7**; memoization computes each **once** (7 values; every other call is an $O(1)$ cache lookup).

(c) Any of: no recursion overhead or stack-depth limit; the fill order is explicit, so you can keep only the rows you need (less memory); the instructor expects bottom-up on the quiz (Campuswire #224).

(d) Let $M = \max(K(i-1, w),\ v_i + K(i-1, w-w_i))$. *At least:* the best set of the first $i-1$ items with capacity w is still feasible, giving $K(i-1, w)$; adding item i to the best set for capacity $w - w_i$ is feasible and gives $v_i + K(i-1, w-w_i)$. So $K(i, w) \geq M$. *At most:* take an optimal set for (i, w). If it omits item i, it uses only the first $i-1$ items within w, so its value is $\leq K(i-1, w)$. If it includes item i, removing it leaves a set of the first $i-1$ items within $w - w_i$, so its value is $\leq v_i + K(i-1, w-w_i)$. Either way $K(i, w) \leq M$.

(e) Row i only reads row $i-1$, so keep two rows of length $n+1$ and swap: $O(n)$ space, still $\Theta(mn)$ time. The traceback needs the whole table to retrace decisions from (m, n) back to $(0, 0)$; once earlier rows are overwritten, you cannot recover the subsequence (only its length).

Rubric (5 pts).

- **+1 pt** (a) overlapping subproblems vs. independent subproblems.
- **+1 pt** (b) 25 calls, 7 distinct, each computed once.
- **+0.5 pts** (c) a valid reason.
- **+1.5 pts** (d) both directions of the argument.
- **+1 pt** (e) two-row trick and why traceback is lost.

Q2. Unbounded knapsack

(a)

$$U(w) = \begin{cases} 0 & w = 0 \text{ (or no type fits)} \\ \max\limits_{i:\, w_i \leq w} \left(v_i + U(w - w_i)\right) & \text{otherwise} \end{cases}$$

Since copies are unlimited, after packing a box of type i the remaining problem is the same problem with less capacity: you don't need to remember which types are "used up" (that's why 0/1 knapsack needs the index i). This is coin change with max instead of min.

(b) Table.

w	0	1	2	3	4	5	6	7	8
$U(w)$	0	0	3	5	6	9	10	12	14

e.g. $U(8) = \max(3 + U(6),\ 5 + U(5),\ 9 + U(3)) = \max(13, 14, 14) = 14$.

(c) Unlimited: $U(6) = \mathbf{10}$ (two type-2 boxes). 0/1: $K(3, 6) = \mathbf{9}$ (one type-3 box; type 1 + type 2 gives only 8). Reusing the type-2 box is allowed only in the unbounded version.

(d)

```
function UNBOUNDED-KNAPSACK (weights, values, n, W):
    U = new array [0..W]
    U[0] = 0
    for w from 1 to W+1:          // index W+1 non-inclusive
        U[w] = 0
        for i from 1 to n+1:      // index n+1 non-inclusive
            if weights[i] <= w:
                U[w] = max(U[w], values[i] + U[w - weights[i]])
    return U[W]
```

$\Theta(nW)$ time, $\Theta(W)$ space.

Rubric (5 pts).

+1.5 pts Recurrence and why one index suffices.

+1.5 pts Table correct (-0.5 per wrong entry, min 0).

+1 pt 10 vs. 9 with explanation.

+1 pt Pseudocode and $\Theta(nW)$.